



Web Search, Feature Dependencies and Multimodal Learning

IFN647 – Week 12

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides	Activities
1	Web search	2-4	
2	Link analysis	5-11	
3	Attention Mechanism for Feature Dependency	12-15	
4	Multi-View Data	16-21	
5	Multimodal Data	22-24	
6	Contrastive Learning	25-35	
	References	36	

1. Web Search

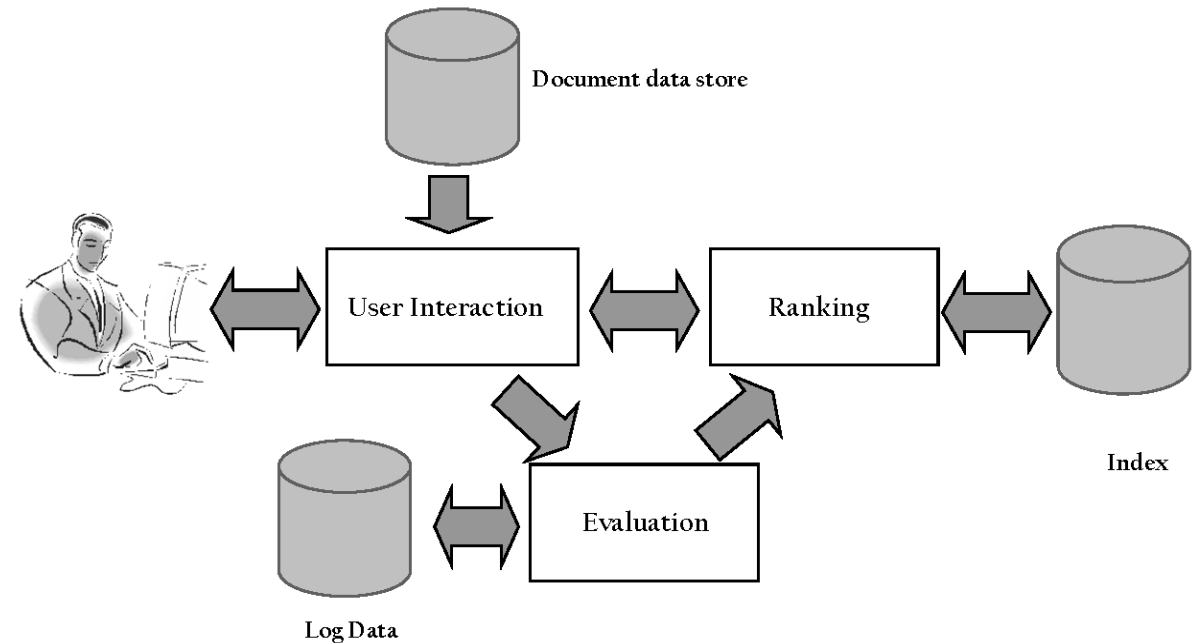
- Web search is the process to analyze and rank webpages based on their relevance to a search query.
- The popular systems to support Web search are search engines, such as Google.
- A search engine is the practical application of information retrieval (IR) techniques to large-scale text collections.
- Some retrieval models focus on topical relevance, but a search engine deployed in a real environment must use ranking algorithms that incorporate user relevance.
- Although web search is by far the most common application involving IR, search is also a crucial part of applications in corporations, government, and many other domains.

- CRICOS No. 00213J



Query Process

- User interaction
 - supports creation and refinement of query, display of results
- Ranking
 - uses query and indexes to generate ranked list of documents
- Evaluation
 - monitors and measures effectiveness and efficiency (primarily offline)



2. Link Analysis

- The Web contains a large number of links connecting pages which are a key component of the Web.
- Link analysis analyses the node and connection structure (document structure) of a web site or extracting patterns from hyperlinks in the web.
- Document parser recognizes structure using markup, such as HTML tags
 - Headers, anchor text, bolded text all likely to be important
 - Metadata and Links can also be important

• Example Web Page

Tropical fish

From Wikipedia, the free encyclopedia

Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species. Fishkeepers often use the term *tropical fish* to refer only those requiring fresh water, with saltwater tropical fish referred to as *marine fish*.

Tropical fish are popular aquarium fish , due to their often bright coloration. In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

```
<html>
<head>
<meta name="keywords" content="Tropical fish, Airstone, Albinism, Algae eater,
Aquarium, Aquarium fish feeder, Aquarium furniture, Aquascaping, Bath treatment
(fishkeeping),Berlin Method, Biotope" />
...
<title>Tropical fish - Wikipedia, the free encyclopedia</title>
</head>
<body>
...
<h1 class="firstHeading">Tropical fish</h1>
...
<p><b>Tropical fish</b> include <a href="/wiki/Fish" title="Fish">fish</a> found in <a
href="/wiki/Tropics" title="Tropics">tropical</a> environments around the world,
including both <a href="/wiki/Fresh_water" title="Fresh water">freshwater</a> and <a
href="/wiki/Sea_water" title="Sea water">salt water</a> species. <a
href="/wiki/Fishkeeping" title="Fishkeeping">Fishkeepers</a> often use the term
<i>tropical fish</i> to refer only those requiring fresh water, with saltwater tropical fish
referred to as <i><a href="/wiki/List_of_marine_aquarium_fish_species" title="List of
marine aquarium fish species">marine fish</a></i>.</p>
<p>Tropical fish are popular <a href="/wiki/Aquarium" title="Aquarium">aquarium</a>
fish , due to their often bright coloration. In freshwater fish, this coloration typically
derives from <a href="/wiki/Iridescence" title="Iridescence">iridescence</a>, while salt
water fish are generally <a href="/wiki/Pigment" title="Pigment">pigmented</a>.</p>
...
</body></html>
```

Hyperlinks and Anchor Text

- Links are a key component of the Web.
- Important for navigation, but also for Web search
 - e.g., `<a href= "http://example.com">Example website</a>`
 - “Example website” is the anchor text
 - “http://example.com” is the destination link
 - both are used by search engines

- Anchor text is used as a description of the content of the *destination page*
 - i.e., collection of anchor text in all links pointing to a page used as an additional text field
- Anchor text tends to be short, descriptive, and similar to query text
- Retrieval experiments have shown that anchor text has significant impact on effectiveness for *some types of queries*.

PageRank

- Billions of web pages, some more informative than others.
- Links can be viewed as information about the *popularity* (*authority?*) of a web page
 - can be used by ranking algorithm.
- *Inlink* (links pointing to a page) count could be used as simple measure.
- Link analysis algorithms like PageRank provide more reliable ratings
 - less susceptible to link spam.

Random Surfer Model

- Assume people browse the Web using the following algorithm:
 - Choose a random number r between 0 and 1
 - If $r < \lambda$:
 - Go to a random page
 - If $r \geq \lambda$:
 - Click a link at random on the current page
 - Start again
- PageRank of a page is **the probability** that the “random surfer” will be looking at that page
 - links from popular pages will increase PageRank of pages they point to

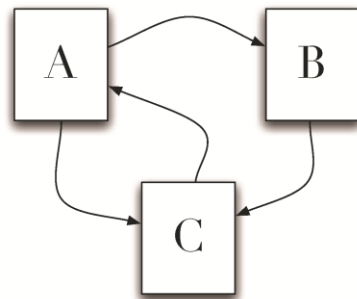
- **Dangling Links**
 - Random jump prevents getting stuck on pages that
 - do not have links
 - contains only links that no longer point to other pages
 - have links forming a loop
 - Links that point to the first two types of pages are called *dangling links*
 - may also be links to pages that have not yet been crawled

PageRank function

- PageRank (PR) of page $PR(C) = PR(A)/2 + PR(B)/1$
- More generally,

$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

- where B_u is the set of pages that point to u , and L_v is the number of outgoing links from page v (not counting duplicate links)
- For the above example, we have $B_C = \{A, B\}$, $L_A = 2$ and $L_B = 1$.



PageRank Example:

- Don't know PageRank values at start
- Assume equal values (1/3 in this case), then iterate:
 - first iteration:
 - $PR(C) = 0.33/2 + 0.33 = 0.5$, $PR(A) = 0.33$, and $PR(B) = 0.17$
 - second:
 - $PR(C) = 0.33/2 + 0.17 = 0.33$, $PR(A) = 0.5$, $PR(B) = 0.17$
 - third:
 - $PR(C) = 0.42$, $PR(A) = 0.33$, $PR(B) = 0.25$
- Converges to $PR(C) = 0.4$, $PR(A) = 0.4$, and $PR(B) = 0.2$

PageRank cont.

- Taking random page jump into account, $1/3$ chance of going to any page when $r < \lambda$
- $PR(C) = \lambda/3 + (1 - \lambda) \cdot (PR(A)/2 + PR(B)/1)$
- More generally,

$$PR(u) = \frac{\lambda}{N} + (1 - \lambda) \cdot \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

- where N is the number of pages, λ typically, is 0.15

PageRank Algorithm

11

```
1: procedure PAGERANK( $G$ )
2:    $\triangleright G$  is the web graph, consisting of vertices (pages) and edges (links).
3:    $(P, L) \leftarrow G$   $\triangleright$  Split graph into pages and links
4:    $I \leftarrow$  a vector of length  $|P|$   $\triangleright$  The current PageRank estimate
5:    $R \leftarrow$  a vector of length  $|P|$   $\triangleright$  The resulting better PageRank estimate
6:   for all entries  $I_i \in I$  do
7:      $I_i \leftarrow 1/|P|$   $\triangleright$  Start with each page being equally likely
8:   end for
9:   while  $R$  has not converged do
10:    for all entries  $R_i \in R$  do
11:       $R_i \leftarrow \lambda/|P|$   $\triangleright$  Each page has a  $\lambda/|P|$  chance of random selection
12:    end for
13:    for all pages  $p \in P$  do
14:       $Q \leftarrow$  the set of pages such that  $(p, q) \in L$  and  $q \in P$ 
15:      if  $|Q| > 0$  then
16:        for all pages  $q \in Q$  do
17:           $R_q \leftarrow R_q + (1 - \lambda)I_p/|Q|$   $\triangleright$  Probability  $I_p$  of being at
            page  $p$ 
18:        end for
19:      else
20:        for all pages  $q \in P$  do
21:           $R_q \leftarrow R_q + (1 - \lambda)I_p/|P|$ 
22:        end for
23:      end if
24:       $I \leftarrow R$   $\triangleright$  Update our current PageRank estimate
25:    end for
26:  end while
27:  return  $R$ 
28: end procedure
```

$$|P| = N$$

$$R-I=0 \text{ or } |R-I| < \epsilon$$

$$Q = \{ q \in P, (p, q) \in L \}$$

$$\text{for } (p, q) \in L$$

$$|P| \gg |Q|$$

3. Attention Mechanism for Feature Dependency

- Overview of Bag of Words
 - “Bag of Words” representation means a simple representation of text data, where a document is considered to be an unordered collection of words (terms) with no relationships, either syntactic or statistical, between them.
 - Many of models discussed before, such as the BM25 model or vector space models are based on a bag of words representation.
 - It has been very successful in retrieval experiments and web search compared to more complex representations of text content.
 - From a linguistic point of view, “bag of words” is extremely limited since no one could read a “bag of words” representation and get the same meaning as the original normal text.
 - In some cases, incorporating simple phrases and word proximity into a word-based representation, which would seem to have obvious benefits.
- Properly modeling **term dependencies** (the relationship between terms) is possible but typically more difficult.
- Generative models (e.g., the Naïve Bayes classifier) adopt simplifying probabilistic assumptions, such as conditional independence, to mitigate the complexity of modeling term dependencies; despite this, they frequently demonstrate strong robustness and effectiveness in practice.

Attention Equation

- An attention mechanism lets each feature:
 - look at other features and decide how important they are
- So instead of treating features independently, it learns dependencies dynamically.
- Basic attention equation for a feature:
 - $\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T \frac{QK^T}{\sqrt{d_k}}\right) V$
 - Where, the feature (term) is turned into three vectors:
 - **Query (Q)** $\rightarrow$ what I'm looking for, e.g., feature (term) 'sat'
 - **Keys (K)** $\rightarrow$ what I contain, all terms, e.g., 'The', 'cat', 'sat', 'on', 'the', 'mat'
 - **Value (V)** $\rightarrow$ what I offer, e.g., same as keys
 - And d_k is the dimension (length) of the key vectors (or query vectors)

Example: for “sat” in: “*The cat sat on the mat*”

- Assume each word is represented in a 3D space (for simplicity):
- Let $Q = [1, 1, 2]$ (for term ‘sat’), we have
- $K^T = [[1, 0, 1, 0, 1,], [0, 2, 1, 1, 0, 2], [1, 0, 2, 1, 0, 1]]$ and $V = K$, $d_k = 3$

- Computer dot product using matrix multiplication:

$$(1, 1, 2) \begin{pmatrix} 1, 0, 1, 0, 1, 0 \\ 0, 2, 1, 1, 0, 2 \\ 1, 0, 2, 1, 0, 1 \end{pmatrix} = (3, 2, 6, 3, 1, 4)$$

- Scale by $\sqrt{3}$: $(3, 2, 6, 3, 1, 4) = (1.73, 1.15, 3.46, 1.73, 0.58, 2.31)$
- Softmax* (approx): $(1.73, 1.15, 3.46, 1.73, 0.58, 2.31) = (0.097, 0.054, 0.547, 0.097, 0.031, 0.173)$ # 0.547 means a strong self-attention

- Weighted sum of V

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \begin{matrix} V \\ \begin{pmatrix} 1, 0, 1 \\ 0, 2, 0 \\ 1, 1, 2 \\ 0, 1, 1 \\ 1, 0, 0 \\ 0, 2, 1 \end{pmatrix} \end{matrix} = (0.675, 1.098, 1.461)$$

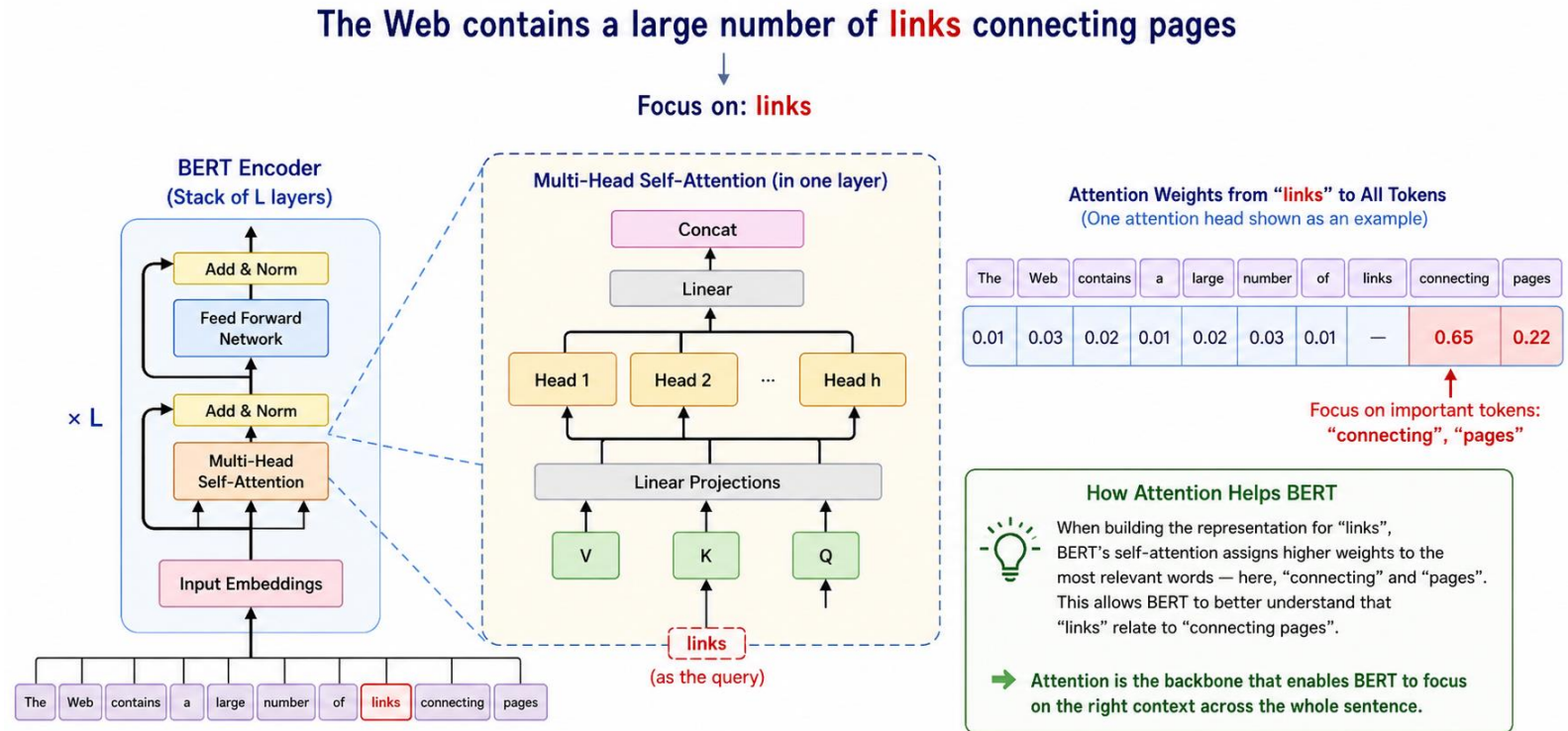
Word (feature)	Vector
The	[1, 0, 1]
cat	[0, 2, 0]
sat	[1, 1, 2]
on	[0, 1, 1]
the	[1, 0, 0]
mat	[0, 2, 1]

$$K = \begin{pmatrix} 1, 0, 1 \\ 0, 2, 0 \\ 1, 1, 2 \\ 0, 1, 1 \\ 1, 0, 0 \\ 0, 2, 1 \end{pmatrix}$$

Attention to BERT

15

- BERT embeddings and the BERT encoder are related, but they play very different roles in the model.
 - **Embeddings: Converting words into vectors**, each token (word/subword) is converted into a dense numerical vector.
- BERT Encoder: Understanding context
 - It is the **deep neural network** that processes those embeddings, contains multiple Transformer encoder layers with:
 - Self-attention
 - Feed-forward neural networks
 - Residual connections
 - Layer normalization
 - It transforms the initial embeddings into **contextualized representations**.

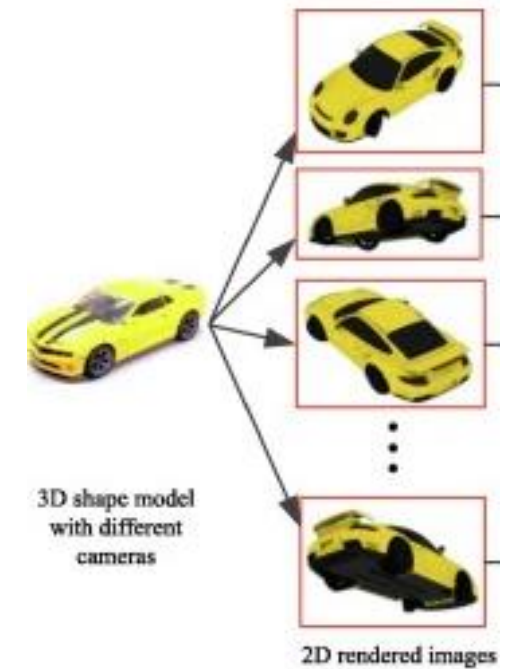


4. Multi-View Data

- Multi-view learning is an emerging deep learning method to combine different views into a single view to improve the generalization ability of models, including hybrid models.
- In the literature, multi-view learning is also referred to as [data fusion](#) from multi-view data.
- For example,
 - For imaging data processing, it uses three steps: (1) project a 3D image into multiple views; (2) extract a view function for each view; (3) fuse these view functions to accurately classify 3D images.
- Co-training
 - It is a *semi-supervised learning* technique that requires two distinct views of the data (i.e., two different feature sets), which are ideally conditionally independent given the class label.
 - Initially, a separate classifier is trained on each view using the available labelled data.
 - During each iteration, each classifier selects its most confident predictions on the unlabelled data and uses them as pseudo-labelled examples to augment the training set of the other classifier. This process is repeated iteratively to improve performance.

CRICOS No.00213J

Multi-View Data Example:
Each view represents a different observation of the same data.



Multiple Feature Sets

- Multi-view data in NLP means representing the *same text* using multiple different feature sets (views), where each view captures a different aspect of the data.
- In previous lectures we have discussed a number of representation features and how they can be used in ranking.
 - **Lexical View (Surface features):** Bag of Words, TF-IDF, N-grams
 - **Semantic View:** Word embeddings (Word2Vec), Sentence embeddings (BERT)
 - **Syntactic View:** Part-of-Speech (POS) tags
 - **Topic View:** LDA latent topic distributions
 - **Metadata View:** Author, Timestamp, etc.
- We also discuss possible machine learning methods to use term dependencies, such as, word embeddings, language models (e.g., n-grams) and probabilistic models.
- However, many applications (e.g., web search) have evolved beyond the stage where a bag of words representation of documents or queries would be adequate.
 - For these applications, representations and ranking based on many different features are required.
 - Features derived from the bag of words are still important, but linguistic, structural, metadata, word embedding, and non-textual content features can also be used effectively.

Linear Feature-Based Models

- Bag of words models assume there is no relationship between words, so we first look at how term dependencies can be captured and used in a linear feature-based model.
- Primitive textual features:
 - Term occurrence/nonoccurrence: whether or not a term occurs within a document
 - Term frequency: the number of times a term occurs within a document
 - Inverse document frequency: inverse of the proportion of documents that contain a given term
 - Document length: number of terms within the document
 - Term proximity: occurrence patterns of terms within a document
- High-level textual features that are combinations of the primitive textual feature.
 - Examples of these features are the term weighting functions used by BM25, language modeling, and most other popular retrieval models.
- Nontextual features: PageRank, URL depth, document quality, sentiment, query clarity, etc.
- Some feature functions may have hyperparameters. For example, the BM25 model is typically parameterized to have two hyperparameters, k_1 and b .
- In linear feature-based models, hyperparameters are not part of the model, but of the features themselves. Therefore, hyperparameters must either be fixed before training or adapted in some way during training.

Term dependence models

- IR models that make use of term relationships are often called term dependence models as they do not assume that words occur independently of each other.
- They can be incorporated into a number of features that are used as part of the ranking algorithm.
- Effective models require the combination of many pieces of evidence or **features** about a document's potential relevance.
- For example, we may consider
 - words occur in particular document structures, such as section headings or titles, or
 - whether words are related to each other.
 - In addition, evidence such as the date of publication, the document type, or,
 - in the case of web search, the PageRank number will also be important.

Scoring functions

- The scoring function of linear feature-based model

$$S_{\Lambda}(D; Q) = \sum_j \lambda_j \cdot f_j(D, Q) + Z$$

$S_{\Lambda}(D; Q)$ is a scoring function for a document D
and query Q , parameterized by Λ

$f_j(D, Q)$ is a feature function that maps query/document
pairs to real values

Z is a constant that does not depend on D
(but may depend on Λ or Q)

- Some models support non-linear functions, but linear is more common.

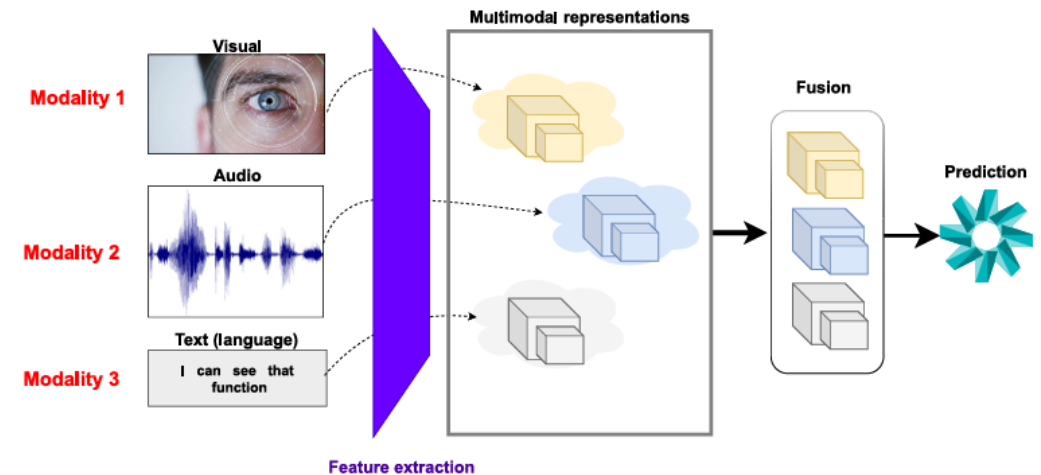
Parameter Setting

- To find best values for parameters
 - need a set of training data T
 - an evaluation function $E(\mathcal{R}_\Lambda; T)$
 - where R_Λ is the rankings produced by the scoring function for all the queries
 - It produces real-valued output given the set of ranked lists and the training data.
 - E is only required to consider the document *rankings* and not the document scores. This is a standard characteristic of the evaluation measures we described before in week 6, such as NDCG.
- Goal of a linear feature-based retrieval model is to find a parameter setting that maximizes E for the training data

$$\hat{\Lambda} = \arg \max_{\Lambda} E(\mathcal{R}_\Lambda; T)$$

5. Multimodal Data

- Multimodal data can be thought of as a combination of different data formats, such as images, text, audio, and time series.
- Multimodal learning can be viewed as the simultaneous analysis of heterogeneous data from multiple sources.
- The ability to represent knowledge at multiple levels of abstraction is one of the most critical challenges in multimodal learning
- Traditional methods
 - **Early fusion:** fusion of low-level features extracted from each modality before making predictions.
 - **Late fusion (also called decision-based fusion):** fusion of features extracted independently from different modalities after predictions are made.
 - **Intermediate fusion:** combination of multimodal features from early and late fusion before decision making.



Multimodal NLP Integration

- It refers to combining **text with other data modalities** (e.g., images, audio, video, graphs) to build models that better understand real-world information.
- Language does not exist in isolation — meaning is enriched when combined with other modalities, e.g.,
 - Image captions
 - Visual question answering
 - News articles with images

Fusion Strategies

- They serve as the core mechanisms for multimodal integration, systematically combining information from diverse data sources to construct a unified, more accurate, and comprehensive representation [5].
- Fusion Strategies
 - *Early Fusion*: Combine features from all modalities at input level; e.g., “concatenate” *text embedding (encoder)* + *image features (encoder)*
 - *Late Fusion*: Process each modality separately and combine predictions at the end.
 - *Hybrid / Intermediate Fusion* (most common today): Modalities interact during learning and Use attention mechanisms to align them, e.g., text attends to image regions, or Image features guided by text.
- Current Approches:
 - Employ machine learning techniques, such as neural networks, attention mechanisms, and contrastive learning, to learn a shared embedding space across modalities and to dynamically assess the relative importance of each modality and its feature-level components.

6. Contrastive Learning

- It trains a model by comparing pairs of similar and dissimilar examples to capture what makes them alike or different.
 - During training, the model is encouraged to pull similar items (e.g., two images of cats) closer together in the representation space, while pushing dissimilar items (e.g., an image of a cat and an image of a toaster) farther apart.
- It can be used for “text attends to image regions,” but typically in combination with attention mechanisms rather than as a direct replacement.
 - Contrastive learning aligns text and image representations, while attention mechanisms enable fine-grained interaction (e.g., text attending to specific image regions). They are often used together.
- Two Flavors of Learning: Supervised vs. Unsupervised
 - Supervised Contrastive Learning
 - Here, labels guide the model training, simplifying the generation of positive and negative pairs.
 - The challenge lies in managing computational resources and achieving efficient training convergence.
 - Hard pairs and triplets, those with high loss values, are crucial for model effectiveness.
 - Unsupervised Contrastive Learning
 - This method leverages properties of data to generate pseudo-labels in the absence of actual labels.
 - A popular example is SimCLR (Simple framework for Contrastive Learning of visual Representations), which creates positive image pairs through random transformations.

- It is a self-supervised learning method introduced by Google Research in 2020.
- It's widely used for learning strong image representations *without labeled data*.
- Challenges: Requires large batch sizes; Sensitive to augmentations and Needs good negative samples
- **Step-by-step:**
 - **Data Augmentation**
 - Random crop, color jitter, blur, flip, etc.
 - Produce two correlated views: x_i, x_j . # $x_i = \text{augment}(x), x_j = \text{augment}(x)$
 - **Encoder Network (e.g., CNN)**
 - Maps images to representations: # $h_i = \text{encoder}(x_i), h_j = \text{encoder}(x_j)$
 - **Projection Head**
 - Small MLP that maps representations to latent space: # $z_i = \text{projection_head}(h_i), z_j = \text{projection_head}(h_j)$
 - **Contrastive Loss (NT-Xent):** $\text{loss} = \text{contrastive_loss}(z_i, z_j)$ # InfoNCE
 - Pulls positive pairs together
 - Pushes apart negatives (other images in batch)

Unsupervised Contrastive Learning for text

- Often, we don't have nicely labeled text data. There is much more unlabeled text data than labeled data available in the world.
- Contrastive loss
 - It takes the output of the network for a positive example and calculates its distance to an example of the same class and contrasts that with the distance to negative examples.
 - The loss is low if positive samples are encoded to similar (closer) representations and negative examples are encoded to different (farther) representations.

Positive pairs (critical input structure)

- Each sample is transformed into **two related views**:
 - (x_i, x_i^+)
 - E.g., in text, we can dropout noise of a sentence and get a different augmentation of the same sentence.
 - So, the input becomes:
 - $(x_i, \text{augment1}(x_i)) \rightarrow$ positive pair
- Negative samples are all other sentences in a batch $(x_j, j \neq i)$.
- They act as **contrasts**.

Batch structure (practical input)

- Batch structure refers to how training data is organized and grouped into batches before being fed into a model during training.

- A batch usually looks like:

$$\text{Batch} = \{ \begin{array}{l} (x_1, x_1'), \\ (x_2, x_2'), \\ \dots \\ (x_N, x_N') \end{array} \}$$

- So input size is often:
 - N original samples
 - $2N$ augmented views

Contrastive Learning Algorithm

- **Input:**
 - A set of samples (e.g., Text sentences), usually with **augmentations or paired views**
- **Output:**
 - A trained encoder that produces **meaningful embeddings**
- Contrastive loss looks suspiciously like the *softmax* function.

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

- Where
 - $\mathbf{z}_i, \mathbf{z}_j$ embeddings of a positive pair
 - $\text{sim}(\cdot)$: similarity function (usually cosine similarity)
 - τ : temperature (controls sharpness)
 - N : number of samples (1 positive + many negatives)

Example: A small batch

- Assume we have two vectors that are similar and an array (set) of dissimilar ones.
- Let x_1 ("*the weather is very sunny today*") and x_2 ("*weather is sunny today*") be the positive vectors, with x_2 (augmented one) being a slightly modified version of x_1 .
- neg is the array of dissimilar vectors, including $x_3, x_4, \dots, x_8$

```
import numpy as np

x1 = np.array([-0.83483301, -0.16904167,
0.52390721])

x2 = np.array([-0.83455951, -0.16862266,
0.52447767])

neg = np.array([
    [ 0.70374682, -0.18682394, -0.68544673],
    [ 0.15465702,  0.32303224,  0.93366556],
    [ 0.53043332, -0.83523217, -0.14500935],
    [ 0.68285685, -0.73054075,  0.00409143],
    [ 0.66652531,  0.51500986,  0.08494579],
    [ 0.76652431,  0.61500886,  0.18494479]])
```


Example: using dot product to calculate similarity score

- `pos_dot` – the similarity score between x_1 and x_2 (positive).
- `neg_dot` – An array of similarity scores of negative samples to x_1 .

```
pos_dot = x1.dot(x2)
print(pos_dot)
num_neg = len(neg)
neg_dot = np.zeros(num_neg)
for i in range(num_neg):
    neg_dot[i] = x1.dot(neg[i])
print(neg_dot)
```

```
0.999999716600668
[-0.91504053  0.30543542 -0.37760565 -0.44443608 -0.59899175 -0.64698801]
```

Example: Contrastive loss

```
# temp parameter
t = 0.07

# concatenated vector divided by the temp parameter
logits = np.concatenate([pos_dot, neg_dot])/t
print(logits)

#e^x of the values
exp = np.exp(logits)

# we only need to take the log of the positive value over the sum of exp.
loss = - np.log(exp[0]/np.sum(exp))
print(loss)
```

```
[ 14.28571024 -13.07200756  4.3633632  -5.3943664  -6.34908688
 -8.55702494 -9.2426859 ]
4.9068770749816975e-05
```

Contrastive Language-Image Pre-training

- **CLIP** is a technique for training a pair of neural network models, one for image understanding and one for text understanding, using a contrastive objective.
- Start by preparing a large dataset of N image-caption pairs
- Let the outputs from the text and image models be respectively $v_1, \dots, v_N, w_1, \dots, w_N$.
- Two vectors are considered "similar" if their dot product is large.
- CLIP produces **two embeddings**:
 - An **image embedding vector** v_i
 - A **text embedding vector** v_t

Multi-class N-pair loss for CLIP

$$-\frac{1}{N} \sum_i \ln \frac{e^{v_i \cdot w_i / T}}{\sum_j e^{v_i \cdot w_j / T}} - \frac{1}{N} \sum_j \ln \frac{e^{v_j \cdot w_j / T}}{\sum_i e^{v_i \cdot w_j / T}}$$

- Where the 1st component is text to image and the second one is image to text.
- N is used to average for multiple text-image pairs.

References

- [1] Y. Gu, J. Yang and G. -Z. Yang, "Multi-view Multi-modal Feature Embedding for Endomicroscopy Mosaic Classification," *2016 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Las Vegas, NV, USA, 2016, pp. 1315-1323, doi: 10.1109/CVPRW.2016.166.
- [2] Koh, J.Y., Salakhutdinov, R. & Fried, D.. (2023). Grounding Language Models to Images for Multimodal Inputs and Outputs. in *Proceedings of Machine Learning Research*, 202:17283-17300 Available from <https://proceedings.mlr.press/v202/koh23a.html>
- [3] J. Wu, W. Gan, Z. Chen, S. Wan and P. S. Yu, "Multimodal Large Language Models: A Survey," *2023 IEEE International Conference on Big Data (BigData)*, Sorrento, Italy, 2023, pp. 2247-2256, doi: 10.1109/BigData59044.2023.10386743.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186,
- [5] O. Chandraumakantham, N. Gowtham, M. Zakariah and A. Almazyad, "Multimodal Emotion Recognition Using Feature Fusion: An LLM-Based Approach," in *IEEE Access*, vol. 12, pp. 108052-108071, 2024, doi: 10.1109/ACCESS.2024.3425953.
- [6] Chen, T., Kornblith, S., Norouzi, M. & Hinton, G.. (2020). A Simple Framework for Contrastive Learning of Visual Representations. in *Proceedings of the 37th International Conference on Machine Learning*, 119:1597-1607 Available from <https://proceedings.mlr.press/v119/chen20j.html>.